

Rapport Projet - Programmation répartie

Alexis CERDA DE ALMEIDA VILACA

Logan SAGET

Taïno HERMANN

Emelyne FOUSSE

9 juin 2026

Table des matières

1	Introduction	2
2	Le raytracer local	3
2.1	Mesures de temps d'exécution	3
2.2	Reproduction de l'image	4
3	Architecture de l'application répartie	6
3.1	Schéma de l'architecture	6
3.2	Processus fixes et processus mobiles	7
3.3	Données échangées entre les processus	7
4	Fonctionnement des composants	9
4.1	Le service central	9
4.2	Les nœuds de calcul	9
4.3	Le client / dispatcher	9
5	Mise en œuvre du parallélisme	10
5.1	Distribution des tâches en threads	10
5.2	Synchronisation et assemblage des résultats	10
6	Conclusion	11

1 Introduction

Lien du répertoire GIT : https://github.com/HermannT88/projet_rmi_calcul_parallele

Ce projet a pour objectif de concevoir et de mettre en oeuvre une application de calcul parallèle distribué appliquée au rendu d'images par lancer de rayons. La synthèse d'images par cette méthode étant une tâche particulièrement gourmande en ressources CPU, l'approche retenue consiste à diviser l'image globale en une multitude de blocs indépendants. En exploitant l'architecture Java RMI, ces tâches sont distribuées à un ensemble de nœuds de calcul connectés à un service central. Cette parallélisation des données permet de réduire considérablement le temps global d'exécution en mutualisant la puissance de calcul de plusieurs machines connectées en réseau.

2 Le raytracer local

2.1 Mesures de temps d'exécution

Pour réaliser des mesures sur le temps de calcul, nous avons fait une boucle qui exécute 10 fois le calcul de l'image avec à chaque itération `largeur = i * largeur de base` et `hauteur = i * hauteur de base`

```
for (int i = 1; i <= 10; i++) {  
    int l = largeur*i, h = hauteur*i;  
  
    Instant debut = Instant.now();  
    System.out.println("Itération " + i + " Largeur = " + l + " Hauteur = " + h);  
    Image image = scene.compute(x0, y0, l, h);  
    Instant fin = Instant.now();  
  
    long duree = Duration.between(debut, fin).toMillis();  
  
    System.out.println("Temps de calcul :"+duree+" ms");  
}
```

Nous obtenons alors ces résultats :

Itération	Largeur (px)	Hauteur (px)	Temps de calcul (ms)
1	512	512	3182
2	1024	1024	7372
3	1536	1536	13642
4	2048	2048	23977
5	2560	2560	32773
6	3072	3072	48813
7	3584	3584	64370
8	4096	4096	84049
9	4608	4608	107256
10	5120	5120	128906

TABLE 1 – Évolution du temps de calcul en fonction des dimensions

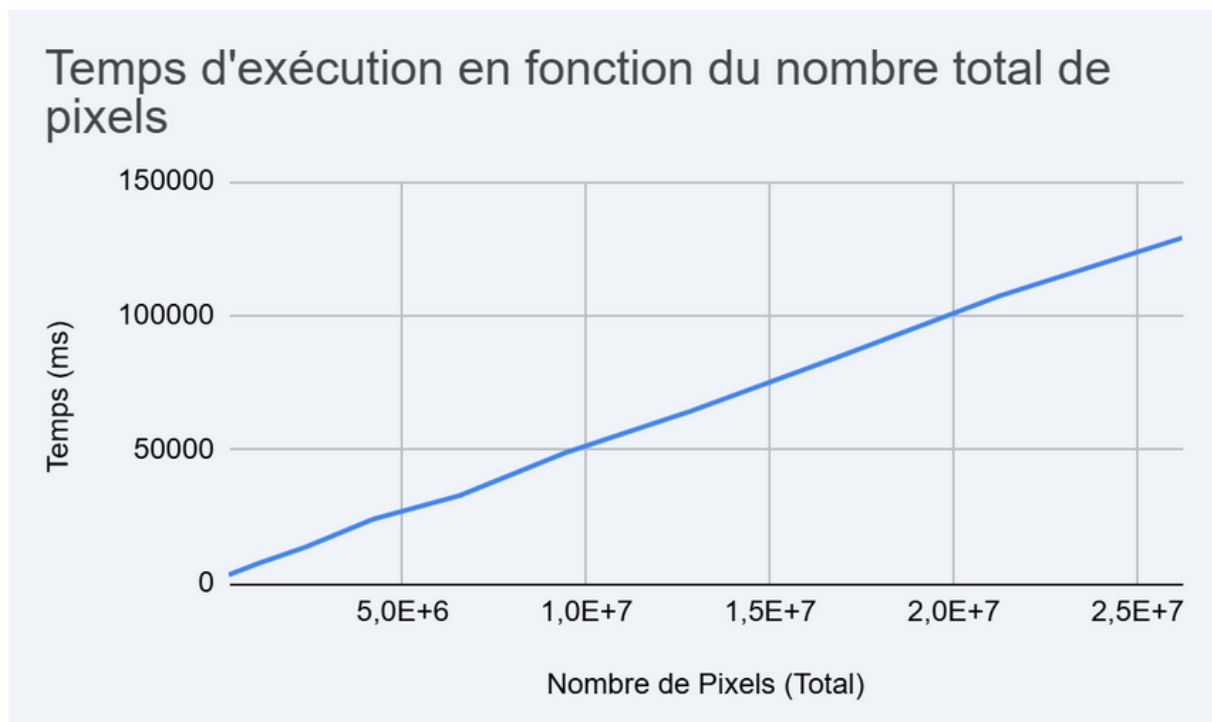


FIGURE 1 – Temps d'exécution en fonction des pixels

Nous constatons que le temps de calcul augmente proportionnellement au nombre de pixels de l'image.

2.2 Reproduction de l'image

L'objectif est de reproduire l'image ci-dessous :

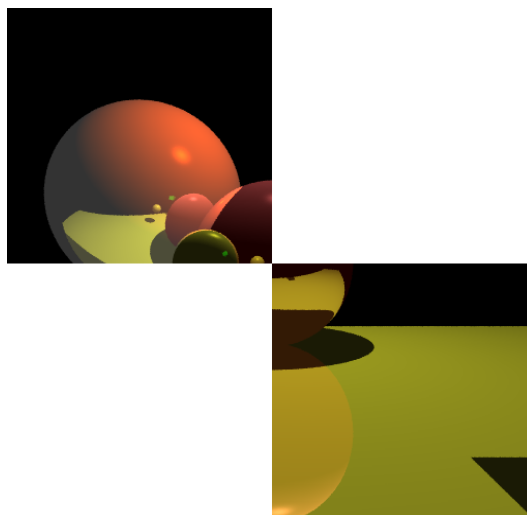


FIGURE 2 – Image à reproduire

Pour reproduire l'image, nous allons calculer individuellement les deux parties de l'image attendue.

La première partie, le programme va calculer l'image en partant des coordonnées 0 ; 0 soit le coin supérieur droit. Puis il va calculer l'image jusqu'à son centre, donc à la moitié de la hauteur et la moitié de la largeur. Ce qui nous donne la ligne suivante :

```
Image image1 = scene.compute(x0, y0, l/2, h/2);
```

Pour la seconde, nous allons recréer une image qui cette fois va commencer au milieu et encore avancé en largeur et en hauteur de la moitié. Nous avons alors cette ligne :

```
Image image2 = scene.compute(l/2, h/2, l/2, h/2);
```

Enfin nous affichons les deux images calculées précédemment sur la même fenêtre, avec la première au coordonnées 0 ; 0 et la seconde au centre de l'image :

```
disp.setImage(image1, x0, y0);  
disp.setImage(image2, l/2, h/2);
```

Pour le code complet, nous arrivons donc à ce résultat :

```
int x0 = 0, y0 = 0;  
int l = largeur, h = hauteur;  
  
// Chronométrage du temps de calcul  
Instant debut = Instant.now();  
System.out.println("Calcul de l'image :\n - Coordonnées : "+x0+", "+y0  
+" \n - Taille "+ largeur + "x" + hauteur);  
Image image1 = scene.compute(x0, y0, l/2, h/2);  
Image image2 = scene.compute(l/2, h/2, l/2, h/2);  
Instant fin = Instant.now();  
  
long duree = Duration.between(debut, fin).toMillis();  
  
System.out.println("Image calculée en : "+duree+" ms");  
  
// Affichage de l'image calculée  
disp.setImage(image1, x0, y0);  
disp.setImage(image2, l/2, h/2);
```

3 Architecture de l'application répartie

3.1 Schéma de l'architecture

Pour notre première idée d'architecture, l'application avait un client qui envoyait le fichier txt correspondant à l'image à calculer au service central. Ce service se serait ensuite chargé du découpage en threads et de la répartition entre les nœuds de calcul. Une fois l'image calculée entièrement, le service central aurait renvoyé l'objet Image au Client.

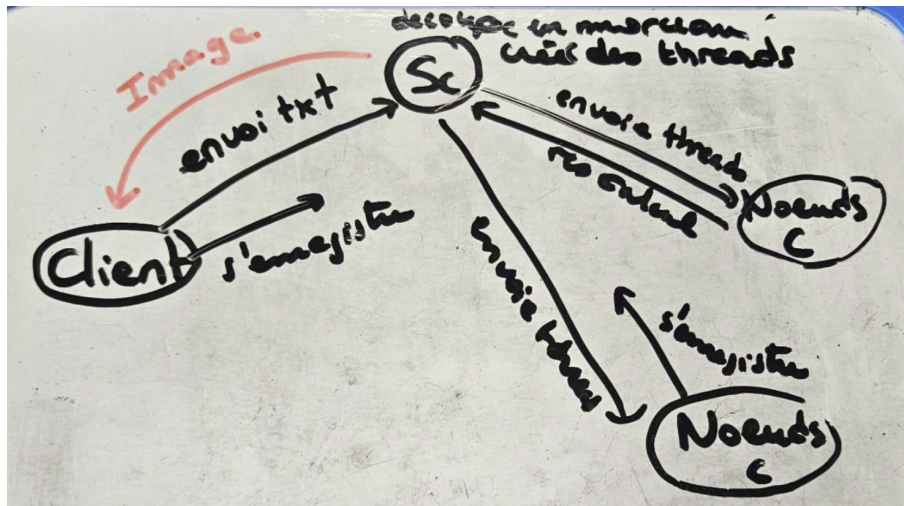


FIGURE 3 – Première version

Pour la deuxième version, le fonctionnement était très similaire à la première version, avec cette fois toute la partie découpage des tâches et échanges avec les nœuds déléguée à un service Répartiteur.

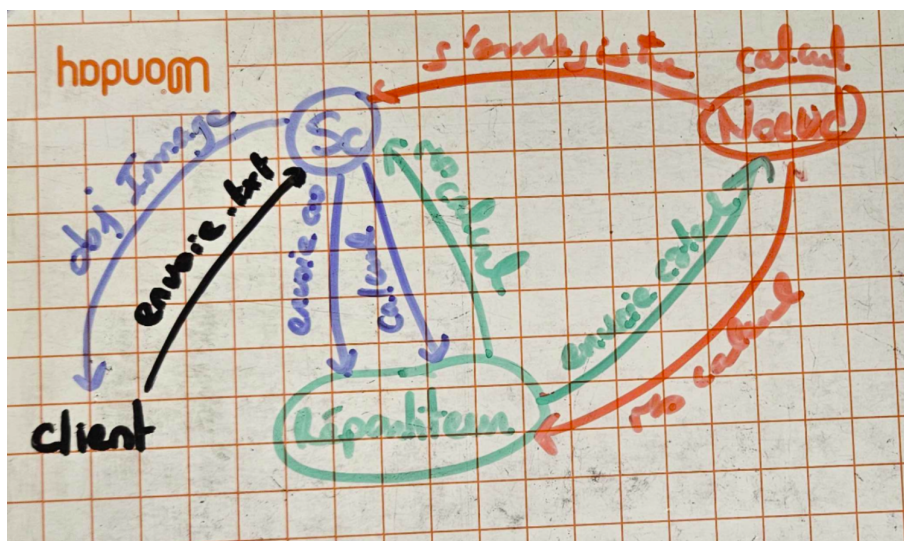


FIGURE 4 – Deuxième version

Dans la version finale, toute la logique de découpage en threads a été redonnée au Client. Le service central ne sert plus qu'à fournir et gérer les nœuds, c'est-à-dire maintenir une liste de

nœuds, en ajouter, en supprimer et envoyer un nœud lorsqu'un Client le demande.

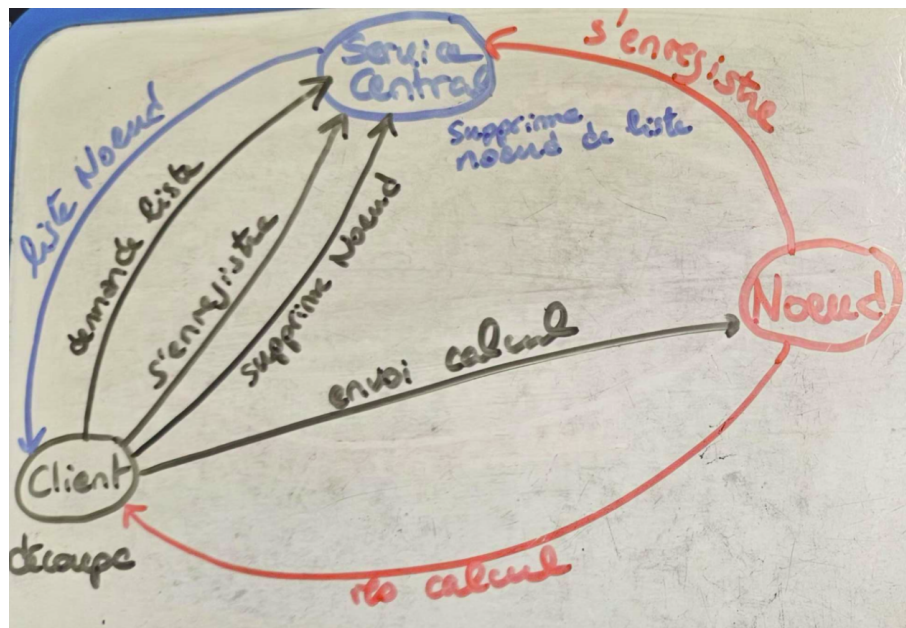


FIGURE 5 – Version finale

* Contrairement à ce qui est indiqué sur le schéma, le service central renvoie un unique nœud au client et non la liste entière.

3.2 Processus fixes et processus mobiles

Dans notre architecture, nous pouvons distinguer deux types de processus en fonction de leur rôle et de leur cycle de vie sur le réseau :

Le processus fixe : Le processus fixe est le Service Central. Il doit être lancé en premier et reste actif de manière permanente. Il écoute sur un port RMI défini (le port 1099 par défaut) et attend que d'autres processus s'y connectent pour tenir à jour son annuaire de nœuds de calcul.

Les processus mobiles : Les processus mobiles apparaissent et disparaissent dynamiquement au cours du temps :

- Les nœuds de calcul (workers) : Ils démarrent, s'enregistrent auprès du service central, et exposent leurs méthodes de calcul sur des ports dynamiques. Ils peuvent se déconnecter ou tomber en panne à tout moment.
- Le client (dispatcher) : Il se connecte de manière éphémère lorsqu'il a une image à calculer. Il interroge le service pour obtenir les nœuds, effectue sa distribution de tâches en parallèle, puis se termine une fois l'image finale assemblée.

3.3 Données échangées entre les processus

Il est important de noter que le service central ne fait pas transiter les calculs d'images. Le client dialogue directement avec les nœuds. Voici les véritables échanges de données s'opérant dans le système :

- Entre les nœuds et le Service Central : Les nœuds s'enregistrent en envoyant au service leur propre référence RMI (un stub) de type `ComputeNode`.

- Entre le Client et le Service Central : Le client demande à récupérer des nœuds de calcul. Le service lui retourne en réponse la référence distante (stub) d'un nœud disponible en appliquant une politique de round-robin.
- Entre le Client et les nœuds de calcul : C'est ici que se fait la transmission du travail lourd. Le client envoie directement aux nœuds le nom du fichier de description (ex : `simple.txt`), les coordonnées de départ du bloc à traiter (x, y) , ses dimensions (w, h) , ainsi que la taille totale de l'image.
- Entre les nœuds de calcul et le Client : Une fois le calcul d'un bloc terminé, le nœud retourne directement au client un objet sérialisable de type **Image** contenant les pixels calculés. Le client se charge ensuite de les assembler pour former le résultat final.

4 Fonctionnement des composants

4.1 Le service central

Le service central agit comme un annuaire dynamique pour les ressources de calcul. Il implémente l'interface `ServiceInterface` et possède les rôles suivants :

- **Registre des nœuds** : Il maintient une liste interne des nœuds de calcul actuellement connectés.
- **Gestion des connexions/déconnexions** : Il expose des méthodes `enregistrerNoeud` et `supprimerNoeud` pour mettre à jour cette liste.
- **Répartition de charge (Round-Robin)** : La méthode `obtenirNoeud()` distribue les nœuds disponibles de manière cyclique. Cela garantit qu'un client ne monopolise pas un seul nœud si plusieurs sont disponibles.

Il est important de noter que le service central ne gère pas directement les calculs ni les données des images ; il ne fait que mettre en relation les clients et les travailleurs.

4.2 Les nœuds de calcul

Les nœuds de calcul sont les travailleurs (*workers*) de l'architecture. Chaque nœud est un programme autonome qui :

- Instancie un objet implémentant l'interface `ComputeNode`.
- S'enregistre auprès du service central via RMI au démarrage.
- Expose une méthode distante `calculerBloc(...)` qui prend en paramètre le fichier de la scène, la position (x, y) du bloc à calculer, et ses dimensions (w, h) .
- Instancie un moteur de *raytracing* en mémoire, calcule les pixels correspondants à son bloc, et renvoie un objet `Image` (sérialisable) contenant le résultat.

4.3 Le client / dispatcher

Le client est le programme qui souhaite rendre une image. Il joue un rôle actif de *dispatcher* :

1. Il se connecte au service central pour connaître les nœuds disponibles.
2. Il divise l'image complète en plusieurs sous-parties indépendantes appelées "blocs" ou "tâches" (par exemple, un quadrillage 10x10).
3. Il place toutes ces tâches dans une file de travail (une liste partagée).
4. Il lance un thread "worker" local pour chaque nœud disponible.
5. Un thread secondaire nommé "watcher" surveille périodiquement le service central pour détecter l'apparition de nouveaux nœuds et lancer des threads supplémentaires si nécessaire.
6. Si un worker vient à mourir ou à se déconnecter, il tue son thread et renvoie le bloc en cours dans la file de travail.

5 Mise en œuvre du parallélisme

5.1 Distribution des tâches en threads

Pour exploiter les nœuds distants, le client génère un parallélisme local via des `Thread` Java. La liste des tâches (`fileTravail`) est gérée de manière synchronisée. Chaque thread local effectue la boucle suivante :

- Il demande un nœud de calcul au service.
- Tant qu'il reste des blocs, il extrait un bloc via une méthode `prendreProchainBloc()`.
- Il effectue l'appel RMI synchrone `calculerBloc` sur son nœud attribué.

L'architecture gère également la tolérance aux pannes : si l'appel RMI échoue (par exemple si un nœud se déconnecte subitement), l'exception `RemoteException` est interceptée. Le client informe alors le service central de supprimer ce nœud, et le bloc inachevé est remplacé dans la file via `remettreBloc(tache)`.

5.2 Synchronisation et assemblage des résultats

L'assemblage des sous-images (`Image` reçues des nœuds) dans l'image finale est géré par la méthode `recevoirResultat`. Cette méthode copie les pixels du bloc reçu vers l'image complète.

Chaque noeuds appel cette méthode à chaque fois qu'ils terminent un bloc et donc l'ajout à l'image finale et actualise l'image en direct.

Pour éviter les problèmes d'accès concurrents lors de la modification de l'image globale et du compteur de blocs restants, le client utilise le mot-clé `synchronized`. De plus, le compteur `blocsRestants` est décrémenté à chaque résultat reçu. Lorsqu'il atteint 0, la méthode déclenche un `notifyAll()` pour réveiller le thread principal qui était en attente (bloqué par un `wait()`), signifiant ainsi la fin du calcul distribué. L'image est également actualisée en temps réel à l'écran via l'interface graphique `Disp`.

6 Conclusion

La mise en place de RMI nous a permis de distribuer les lourds calculs de *raytracing* sur un réseau d'ordinateurs. L'approche choisie se démarque par sa robustesse et sa flexibilité :

- **Scalabilité horizontale** : L'ajout de nouveaux nœuds pendant le calcul est détecté automatiquement, ce qui accélère la résolution de l'image en temps réel.
- **Tolérance aux pannes** : Un ordinateur défaillant n'interrompt pas le rendu. Le bloc perdu est simplement réaffecté à un autre nœud.

En divisant judicieusement le travail en petites tâches et en évitant les verrous bloquants prolongés, le temps de calcul est drastiquement réduit, offrant des performances bien supérieures à un calcul purement local.